

AI Team Intern Assignment — The Audit

Timebox: designed for ~10 focused hours over 5 days. We grade thinking-per-hour, not volume. A sharp 6-hour submission beats a bloated 20-hour one.

Format: take-home + a 30-minute live defense session.

Ground rules (read these twice)

AI tools are allowed and expected. Claude, ChatGPT, Copilot, Cursor — use whatever you want. We use them daily; pretending you won't would be silly. This assignment is designed so that AI *without your own understanding* produces a mediocre submission. Two rules make that work:

1. **The evidence rule.** Every claim you make — "this line is a bug", "this metric is misleading", "this config is faster" — must ship with a minimal experiment demonstrating it: the exact command, the before/after numbers, and one sentence on why the delta proves your claim. **A claimed flaw without measured evidence scores negative points.** Confident hallucination is the one thing we punish harder than being wrong.
2. **The defense.** In the live session you will re-derive your numbers, modify your code on the spot, and answer counterfactuals ("if we changed X, what happens to your conclusion?"). Anything in your submission is fair game. If you can't defend it, don't submit it.

You must also submit `AI_USAGE.md`: a short honest summary of where AI helped you and where it misled you. This is not a trap — "Claude

wrote 80% of my plotting code and confidently invented a bug that didn't exist" is a perfectly good entry. What we're checking is whether *you* know which parts of your submission you understand.

The scenario

A previous intern benchmarked our tokenizer and our serving stack, wrote `REPORT_v0.md`, and left. Leadership is about to make routing and capacity decisions based on it. Your job is to audit the report before that happens.

You are given (in `starter_kit/`):

- `fertility.py` — the script that produced the tokenizer numbers
 - `REPORT_v0.md` — the report and its recommendations
 - `corpus_sample/` — the tiny corpora the intern used (English + Hindi, parallel line-by-line). **These are smoke-test toys, not an eval set.**
 - `bench/model_spec.md` + `bench/bench_log.csv` — serving setup and the load-test log behind Section 2 of the report
-

Part A — The tokenizer audit (50 pts)

A1. Build a real eval corpus (10 pts). The sample corpora are ~10 sentences. Assemble a proper multilingual eval set: at least 4 languages including English, Hindi, and **two Dravidian languages** (e.g. Kannada, Tamil, Telugu, Malayalam). Parallel corpora such as FLORES-200 are a good starting point but the choice is yours — document it. State corpus size, domain, any preprocessing, and one paragraph on what your corpus *cannot* tell you (sample-size and domain caveats are signal, not weakness).

A2. Audit the script and the metric (20 pts). `fertility.py` has problems. Some are code bugs. At least one is a *conceptual* problem:

the code computes exactly what it says, but what it says is the wrong thing to compute. And at least one thing in the script *looks* suspicious but is actually fine. For every flaw you claim, apply the evidence rule: isolate it, measure its effect on the reported numbers, and state the direction and magnitude of the distortion. Flagging the harmless thing as a bug — without evidence — costs you points.

A3. Corrected analysis (12 pts). Recompute the cross-language comparison properly, on your corpus from A1, with **at least two tokenizers** (one of them multilingual/Indic-aware; open ones are fine) and **at least two denominators** (e.g. per whitespace word, per grapheme cluster, per UTF-8 byte, per parallel sentence). Then answer the question that actually matters: *which single number should drive a routing-and-cost decision, and why?* Hint: think hard about what the denominator is supposed to hold constant across languages.

A4. Recommendation memo (8 pts). ≤ 1 page. Your corrected headline numbers, your routing recommendation, the biggest caveat, and the one metric you would monitor in production to catch this analysis being wrong.

Part B — Capacity reconciliation (20 pts)

Using `bench/model_spec.md` and `bench/bench_log.csv`:

B1 (7 pts). From the model spec alone, compute: (a) KV-cache bytes per token, exactly; (b) the approximate maximum number of concurrent 4096-token sequences this GPU can hold. Show the arithmetic. Then check your prediction against the log.

B2 (6 pts). The long-context sweep (prompt 3584) contains a throughput anomaly relative to naive "throughput scales with batch" expectations. Identify it, explain the mechanism using specific rows and columns, and propose one config or deployment change with a *predicted* quantitative effect.

B3 (4 pts). REPORT_v0 Section 2 concludes that longer prompts give

better throughput and that batch 48 will deliver ~3200 tok/s. Both conclusions come from the same misreading of one column. What is it, what is the honest "goodput" of the batch-24 long-prompt row (derive it from the log — there are two independent ways), and what should the report have said?

B4 (3 pts). One paragraph: which single counter or metric from the serving stack would you pull to *confirm* your B2 mechanism, and what value do you expect it to show?

Part C — Decision memo (15 pts)

Scenario: the product team wants our assistant's replies to sound casual and conversational in Hindi, Kannada, Tamil, Telugu, Bengali and Marathi — current outputs are too formal/textbook. Three paths are on the table:

- **(a)** an SFT pass on synthetic "casualized" response pairs
- **(b)** a small (≤ 1 B) inference-time rewriter model after the main model
- **(c)** prompt-engineering only

Constraints: one A100-80GB for 2 weeks, one native-speaker reviewer (Hindi + Kannada only) for 10 h/week, launch review in 3 weeks, no external API budget.

Write a ≤ 1 -page recommendation containing, explicitly labelled: your **assumptions**, **back-of-envelope arithmetic** (data volume, training or serving cost, reviewer throughput), a **success metric with a numeric threshold**, a **kill criterion** (the observation that would make you abandon your chosen path, and by when), and the **first experiment you would run on day 1**. There is no single right answer; there are many wrong ways to reason.

Deliverables

A repo (zip or git link) containing: _____

```
your-submission/  
  NOTEBOOK.md          # chronological lab notebook (see be  
  AI_USAGE.md           #  
  partA/                # code, corpus prep scripts, correct  
  partB/                # calculations + written answers  
  partC/memo.md
```

NOTEBOOK.md is graded, not decorative. It is a chronological log of hypothesis → experiment → result → revision, including dead ends. A notebook with no dead ends and no surprises is, in our experience, a notebook written after the fact.

Defense session (15 pts, and a gate)

30 minutes, screen shared, your repo open. Expect: "re-derive this number", "run your script with this input I'm about to paste", "add this flag live", and counterfactuals. Fabricated evidence discovered at any point — in the repo or in the defense — is an automatic fail regardless of score.

Scoring summary

component	pts
A1 corpus construction & caveats	10
A2 script/metric audit (evidence rule applies)	20
A3 corrected analysis & denominator reasoning	12
A4 recommendation memo	8
B1–B4 capacity reconciliation	20
C decision memo	15

Defense	15
Total	100

Unverified claimed flaws: **-5 each** (A2 floor at 0). Fabricated evidence: fail. Good luck — and remember, we'd rather see three claims you can defend to the death than ten you can't.